

Programlama Dilleri

Dr. Öğr. Üyesi Hayri Volkan Agun

Bilgisayar Mühendisliği Bölümü

Bursa Teknik Üniversitesi

Alt programların uygulanması

- Bu bölümün temel amacı, programlama dillerinde alt programların (fonksiyonlar ve prosedürler) nasıl gerçekleştirildiğini incelemektir.
- Alt programlar, modern yazılım sistemlerinin en temel yapı taşlarından biridir ve programların modüler, okunabilir ve yeniden kullanılabilir olmasını sağlar.
- Alt programların gerçekleştirilmesi sürecinde en kritik konulardan biri subprogram linkage (alt program bağlama mekanizması) dır.
- Bu mekanizma, bir fonksiyonun çağırılması ve tamamlandıktan sonra çağıran programa geri dönmesi sürecinde gerçekleşen tüm işlemleri kapsar.
- Bu kapsamda şu sorulara cevap aranır:
 - Bir fonksiyon çağrıldığında bellekte neler olur?
 - Parametreler nasıl aktarılır?
 - Lokal değişkenler nerede tutulur?
 - Program akışı nasıl kontrol edilir?
 - Recursive (özyinelemeli) çağrılar nasıl mümkün olur?

Subprogram Linkage

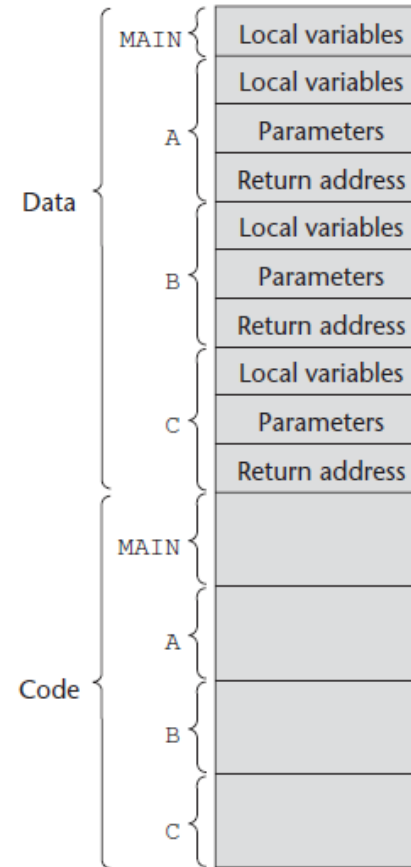
- Subprogram linkage, bir alt programın çağırılması (call) ve tamamlandıktan sonra geri dönmesi (return) süreçlerinin tamamını ifade eder.
- Bir fonksiyon çağırıldığında aşağıdaki işlemler gerçekleştirilir:
 - Parametreler hazırlanır ve aktarılır.
 - Lokal değişkenler için bellek tahsis edilir.
 - Çağıran programın yürütme durumu saklanır:
 - Register içerikleri
 - Program sayacı (PC)
 - CPU durum bitleri
 - Return address (geri dönüş adresi) saklanır.
 - Kontrol çağrılan fonksiyona aktarılır.

Subprogram Linkage

- Subprogram linkage, bir alt programın çağırılması (call) ve tamamlandıktan sonra geri dönmesi (return) süreçlerinin tamamını ifade eder.
- Fonksiyon tamamlandığında:
 - Eğer out veya inout parametre varsa geri kopyalanır.
 - Fonksiyon sonucu uygun yere aktarılır.
 - Lokal değişkenler için ayrılan bellek serbest bırakılır.
 - Çağıran programın durumu geri yüklenir.
 - Kontrol çağıran programa geri verilir.

Basit Alt Programlar

- Basit alt programlar şu özelliklere sahiptir:
 - İç içe (nested) değildir.
 - Lokal değişkenler statiktir (program boyunca sabit yer kaplar).
 - Recursion desteklenmez.
- Bu tür yapılara Fortran'ın ilk sürümleri gösterilebilir.



Basit Alt Programlar

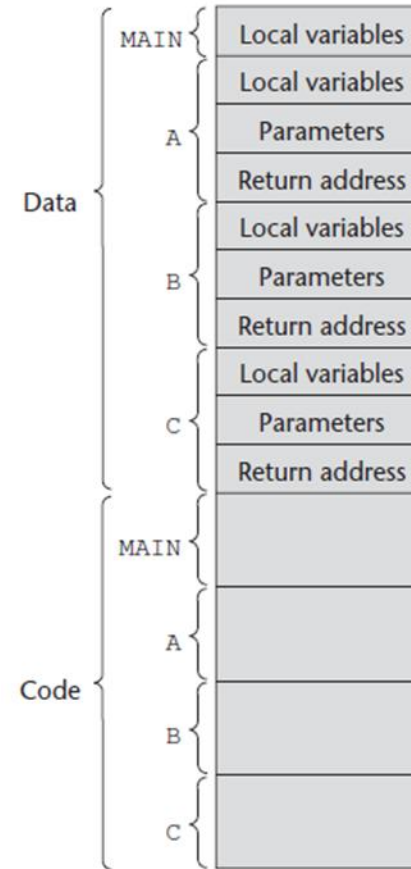
- Basit alt programlar şu özelliklere sahiptir:
 - İç içe (nested) değildir.
 - Lokal değişkenler statiktir (program boyunca sabit yer kaplar).
 - Recursion desteklenmez.
- Bu tür yapılara Fortran'ın ilk sürümleri gösterilebilir.

```
<> C  
  
void A() {  
    int x = 5;  
}
```

- Bu örnekte: x değişkeni program boyunca aynı bellek alanında tutulur.
- Fonksiyon her çağrıldığında yeni bir bellek tahsisi yapılmaz.

Aktivasyon Kaydı

- Bir alt programın çalışması sırasında ihtiyaç duyduğu tüm bilgilerin tutulduğu veri yapısına activation record (AR) denir.
- Bir activation record genellikle şunları içerir:
 - Parametreler.
 - Return address.
 - Lokal değişkenler.
 - Geçici (temporary) veriler.



Basit Alt Programlarda:

- Yapısı derleme zamanında belirlenir.
- Boyutu sabittir.
- Her alt program için sadece bir tane activation record vardır.

Linker ve Programın Oluşturulması

- Derleme süreci genellikle ayrı ayrı yapılır. Örneğin;

```
«» C

// main.c
void A();
int main() {
    A();
}
```

```
«» C

// A.c
void A() {}
```

- Bu dosyalar ayrı ayrı derlenir. Sonrasında **Linker** devreye girer.
- **Linker'in Görevi**
 - Tüm modülleri birleştirmek.
 - Fonksiyon çağrılarını gerçek adreslere bağlamak.
 - Kütüphaneleri programa dahil etmek.

Stack-Dynamic Model

- Lokal değişkenler derleme zamanında değil, çalışma zamanında (runtime) oluşturulur.
- Bellek tahsisi ve serbest bırakılması otomatik olarak yapılır.
- Alt program çağrıları stack (yığın) üzerinden yönetilir.
- Recursion (özyineleme) desteği vardır.
 - Aynı fonksiyonun birden fazla aktif örneği olabilir.
 - Her çağrı kendi bağımsız değişkenlerine sahip olur.
- Stack-dynamic model, basit (static) modele göre daha karmaşıktır:
 - Lokal değişkenlerin dinamik olarak allocate/deallocate edilmesi gerekir.
 - Aynı fonksiyonun birden fazla activation'ı olabilir.
 - Activation record sayısı sabit değildir.

Stack-Dynamic Model

- Stack-dynamic modelde Activation record formatı derleme zamanında bellidir. Ancak boyutu dinamik olabilir.

```
void sub(float total, int part) {  
    int list[5];  
    float sum;  
    . . .  
}
```

Local variables
Parameters
Dynamic link
Return address

↑
Stack top

Local	sum
Local	list [4]
Local	list [3]
Local	list [2]
Local	list [1]
Local	list [0]
Parameter	part
Parameter	total
Dynamic link	
Return address	

Stack-Dynamic Model

- Bir fonksiyon çağrıldığında:
 - Yeni activation record oluşturulur.
 - Parametreler yerleştirilir.
 - Return address eklenir.
 - Dynamic link ayarlanır.
 - EP güncellenir.
- Environment Pointer (EP)EP:
 - Aktif activation record'un başlangıcını gösterir.
 - Tüm değişken erişimleri EP üzerinden yapılır.

Local	sum
Local	list [4]
Local	list [3]
Local	list [2]
Local	list [1]
Local	list [0]
Parameter	part
Parameter	total
Dynamic link	
Return address	

Stack-Dynamic Model

- **Çağrı sırasında:**
 - Yeni AR oluşturulur.
 - Eski EP → dynamic link olarak kaydedilir.
 - EP → yeni AR olarak güncellenir.
- **Dönüş sırasında:**
 - EP → eski dynamic link okunur.
 - Stack pointer geri alınır.

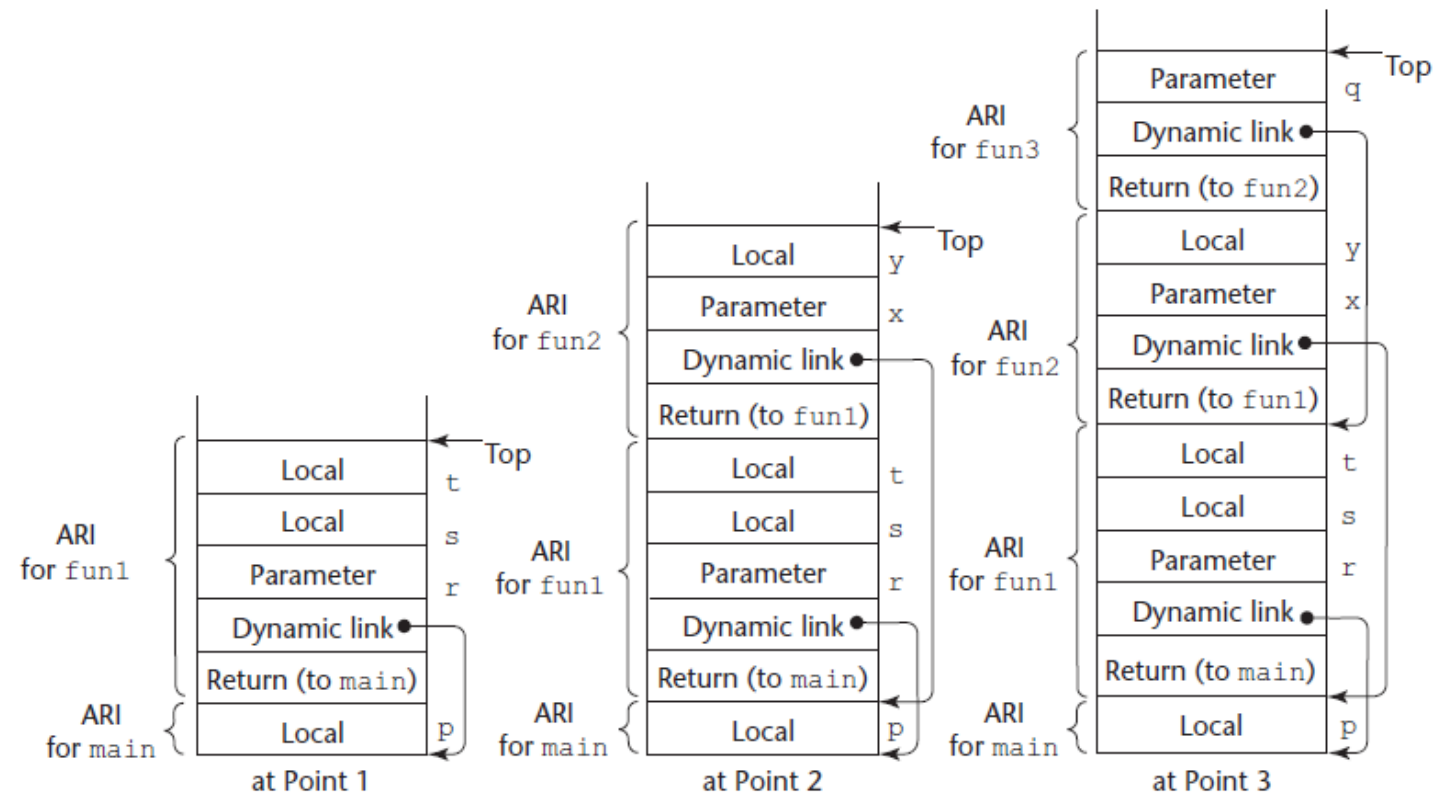
Stack Dynamic Model

- Yaşam döngüsü:
 - Alt Program Çağrıldığı anda aktif olur.
 - Bittiğinde Scope yok olur.
 - Referencing environment silinir.
 - Activation record silinir.
- Caller (Çağırın) İşlemleri
 - Activation record oluşturur.
 - Execution status'u saklar.
 - Parametreleri hesaplar.
 - Return address ekler.
 - Kontrolü aktarır.
- Çağrılan İşlemleri Başlangıç
 - Fonksiyon başında:
 - Eski EP saklanır. (dynamic link)
 - Yeni EP oluşturulur
 - Lokal değişkenler bellek üzerinde ayrılır.
- Çağrılan İşlemleri Bitiş
 - Parametreler geri aktarılır.
 - Return değeri gönderilir.
 - Stack pointer güncellenir.
 - EP geri yüklenir.
 - Kontrol geri verilir.

Stack Dynamic Model

```
void fun1(float r) {  
    int s, t;  
    . . . <----- 1  
    fun2(s);  
    . . .  
}  
  
void fun2(int x) {  
    int y;  
    . . . <----- 2  
    fun3(y);  
    . . .  
}  
  
void fun3(int q) {  
    . . . <----- 3  
}  
  
void main() {  
    float p;  
    . . .  
    fun1(p);  
    . . .  
}
```

main calls fun1
fun1 calls fun2
fun2 calls fun3



Dinamik Zincir

- Çağrı Zinciri stack üzerinde her activation record içinde bulunan dynamic link, bir önceki (çağıran) fonksiyonun activation record'unu gösterir.
- Bu linklerin oluşturduğu yapı Dynamic Chain (veya Call Chain) olarak adlandırılır.
- Programın çalışma anındaki çağrı geçmişini (execution history) temsil eden bağlı yapıdır.
- Başka bir deyişle: "Buraya nasıl geldik?" sorusunun cevabıdır.
- Runtime'da call stack'in mantıksal görünümüdür.
- Bir hata oluştuğunda:
 - Stack trace çıkarılır. Bu, dynamic chain üzerinden yapılır.

Öz Yineleme (Recursion)

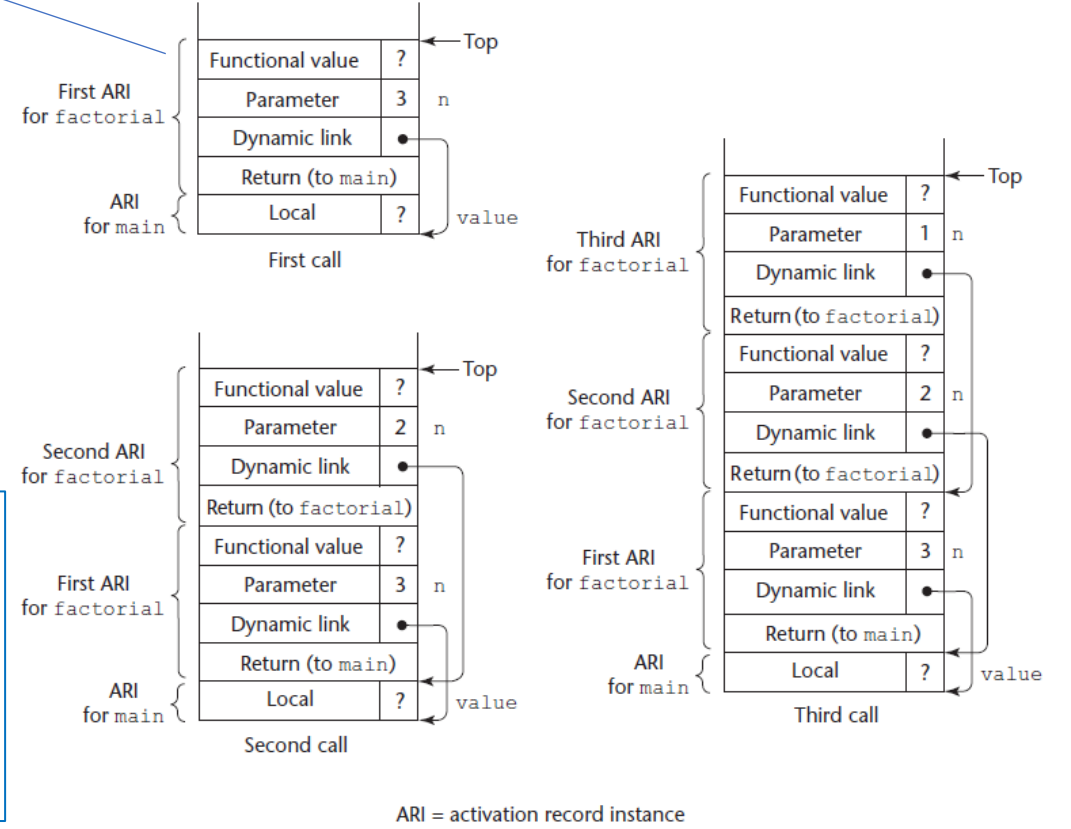
```
int factorial(int n) {  
    <----- 1  
    if (n <= 1)  
        return 1;  
    else return (n * factorial(n - 1));  
    <----- 2  
}  
void main() {  
    int value;  
    value = factorial(3);  
    <----- 3  
}
```

1. Bu örnekte önemli bir fark vardır. Activation record içinde return value için ek alan bulunur.

2. Recursive çağrılar birbirini bekler.

Her çağrı kendi sonucunu saklamak zorundadır. Eğer saklanmazsa değer kaybolur ve doğru hesap yapılamaz.

3. Dikkat edilmesi gereken nokta: İlk başta hiçbir çarpma yapılmaz. Tüm çağrılar stack'e eklenir. Hesaplama geri dönüşte yapılır.



Nested Subprograms

- C-benzeri dillerin aksine bazı diller:İç içe fonksiyon tanımına izin verir:
 - Fortran 95+
 - Ada
 - Python
 - JavaScript
 - Ruby
 - Lua
 - Fonksiyonel diller.
- Ancak bu durumda yeni bir problem ortaya çıkar:Bir fonksiyon, kendi scope'u dışında ama global olmayan bir değişkene nasıl erişecek?

```
<> Python  
  
def f1():  
    x = 10  
  
    def f2():  
        def f3():  
            print(x)    # ← nonlocal variable
```

Burada: x, f3 içinde tanımlı değil global de değil ama erişilebilir.

Nested Subprograms

- Ancak bu durumda yeni bir problem ortaya çıkar: Bir fonksiyon, kendi scope'u dışında ama global olmayan bir değişkene nasıl erişecek?
- **İki Adımlı Erişim Mekanizması**
Nonlocal variable erişimi:
 - **Adım 1: Doğru Activation Record'u Bul**
 - Hangi fonksiyonun değişkeni.
 - **Adım 2: O AR içinde değişkeni bul**
 - `local_offset` kullanılır.
- Activation Record'a Yeni Alan: Static Link
- Nested yapı desteklemek için AR genişletilir:
 - [Return Address]
 - [Static Link] ← YENİ
 - [Dynamic Link]
 - [Parameters]
 - [Local Variables]
- Statik Link: Bir fonksiyonun activation record'undan, onun **lexical parent'ının activation record'una** pointer. Static link'lerin oluşturduğu zincir **Static Chain**.

Statik Chain bs Dynamic Chain

Özellik	Dynamic Chain	Static Chain
Bağlantı	Çağırın fonksiyon	Lexical parent
Amaç	Return / çağrı geçmişi	Nonlocal variable erişimi
Belirlenme zamanı	Runtime	Compile-time + runtime

Statik Chain ile Nonlocal Erişim

Bir değişkene erişmek için: Static chain üzerinde yukarı çık Doğru activation record'u bul local_offset ile değişkene eriş.

Statik Chain vs Dinamik Chain

- f3 içinde x erişimi:
 - f3 depth = 3
 - f1 depth = 1
 - $\text{chain_offset} = 3 - 1 = 2$
- Yani: static chain'de 2 link yukarı çık.
- Dynamic chain: “Kim kimi çağırdı?”
- Static chain: “Kim kimin içinde tanımlı?”

Python

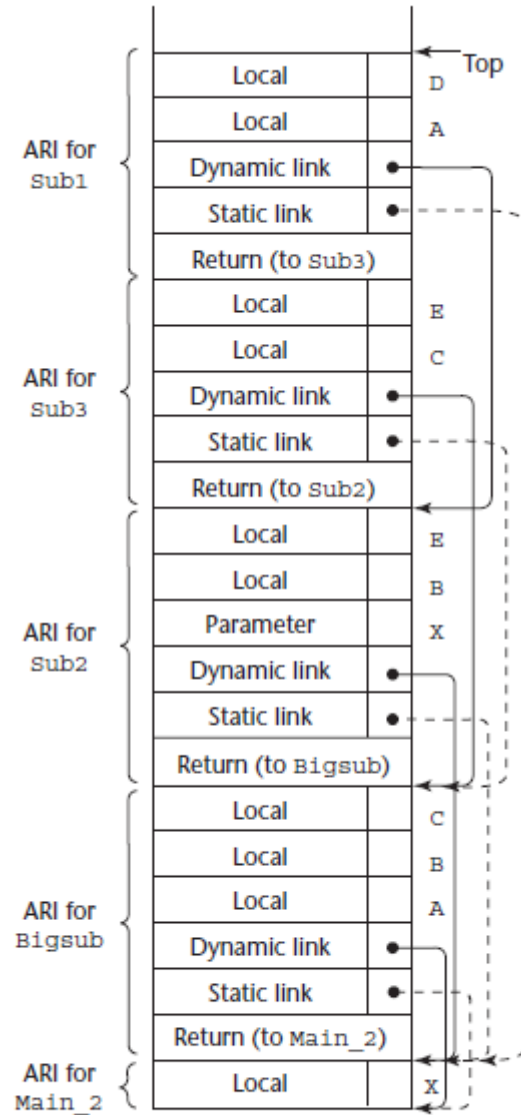
```
def f1():  
    x = 10  
  
    def f2():  
        def f3():  
            print(x)  # ← nonlocal variable
```

Adım 1:

```
current AR (f3)  
→ static link → f2  
→ static link → f1  ✓ bulundu
```

Adım 2:

```
adres = EP(f1) + local_offset(x)
```



ARI = activation record instance

Main_2 calls Bigsub
Bigsub calls Sub2
Sub2 calls Sub3
Sub3 calls Sub1

- Sub1 tamamlandıktan sonra aktivasyon kaydı yığından kaldırılır ve kontrol Sub3'e döner.
- Sub3'teki 2 numaralı konumda:
 - E yereldir $\rightarrow (0, 4)$
 - B, Sub2'de tanımlıdır (en yakın statik ata) $\rightarrow (1, 4)$
 - A, Bigsub'tadır $\rightarrow (2, 3)$
- Sub3 bittikten sonra yığından kaldırılır. Sub2'de 3. konumda:
 - A $\rightarrow$ Bigsub'tan $\rightarrow (1, 3)$
 - D $\rightarrow$ hiçbir kapsamda yok $\rightarrow$ **statik hata**
 - E $\rightarrow$ yerel $\rightarrow (0, 5)$
- Özetle A değişkenine erişimler:
 - $(0, 3) \rightarrow$ yerel
 - $(2, 3) \rightarrow$ iki seviye yukarı
 - $(1, 3) \rightarrow$ bir seviye yukarı

```

procedure Main_2 is
  X : Integer;
  procedure Bigsub is
    A, B, C : Integer;
    procedure Sub1 is
      A, D : Integer;
      begin -- of Sub1
        A := B + C; 1
        ...
      end; -- of Sub1
    procedure Sub2(X : Integer) is
      B, E : Integer;
      procedure Sub3 is
        C, E : Integer;
        begin -- of Sub3
          ...
          Sub1;
          ...
          E := B + A; 2
        end; -- of Sub3
      begin -- of Sub2
        ...
        Sub3;
        ...
        A := D + E; 3
      end; -- of Sub2
    begin -- of Bigsub
      ...
      Sub2(7);
      ...
    end; -- of Bigsub
  begin -- of Main_2
    ...
    Bigsub;
    ...
  end; -- of Main_2

```

Blok

- Birçok programlama dili (özellikle C tabanlı diller), kullanıcı tanımlı yerel kapsamlar (local scope) oluşturmayı sağlar. Bu yapılara blok (block) denir.
- temp değişkeni sadece bu blok içinde tanımlıdır. Blok dışından erişilemez.
- Bloklar isim çakışmasını önler.
- Global/üst scope değişkenler etkilenmez
- Daha güvenli ve modüler kod yazılır

```
«» C  
  
{  
    int temp;  
    temp = list[upper];  
    list[upper] = list[lower];  
    list[lower] = temp;  
}
```

Blok

- Bloklar iki farklı şekilde gerçekleştirilebilir:
 - Yöntem 1: Static Chain (Alt Program Gibi)
 - Bloklar: Parametresiz bir alt program (subprogram) gibi ele alınır.
 - Özellikler: Her blok için bir activation record (aktivasyon kaydı) oluşturulur.
 - Her çalıştırıldığında stack'e eklenir
 - Sonuç: Nested subprogram mantığıyla aynıdır.
 - Daha genel ama biraz maliyetli
 - Yöntem 2: Statik Bellek Yerleşimi (Daha Verimli)
 - Bu yöntem daha basit ve performanslıdır.
 - Programdaki bloklar metinsel sırayla (textual order) çalışır.
 - Aynı anda ihtiyaç duyulan maksimum bellek compile-time'da hesaplanabilir.
 - Bu sayede tek bir alan ayrılır. Blok değişkenleri bu alanı paylaşır.

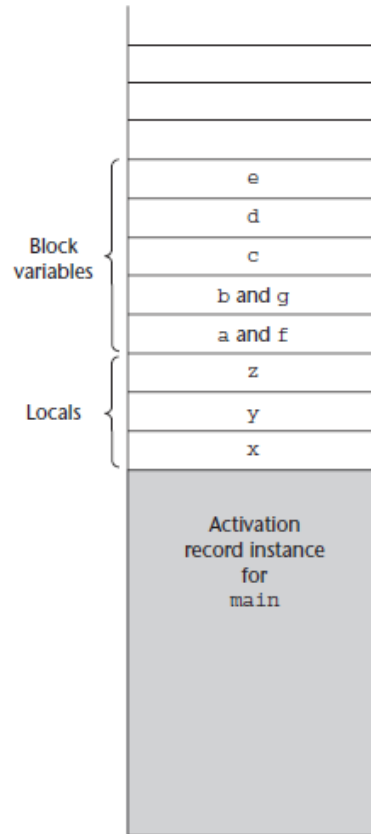
Blok

```
// C
void main() {
    int x, y, z;

    while (...) {
        int a, b, c;

        while (...) {
            int d, e;
        }

        while (...) {
            int f, g;
        }
    }
}
```



- Bellek Kullanımı (Önemli Nokta)
- a, b, c ve f, g aynı anda aktif değildir.
- Aynı bellek alanını paylaşabilirler
 - a, b → ilk while için
 - d,e,f, g → ikinci while içinde
- Bu yüzden: a, b $\leftrightarrow$ f, g (aynı memory alanı)
- Çünkü bloklar strict textual order ile çalışır. Yani bir blok bitmeden diğeri başlamaz. Stack davranışı deterministiktir.

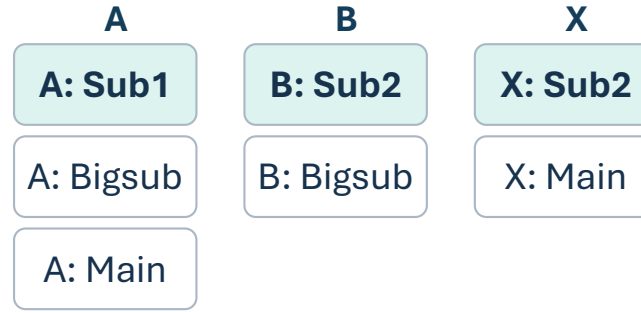
Dinamik Skop

- Dynamic scoping (dinamik kapsam) kullanılan dillerde, nonlocal değişkenlere erişim iki farklı yöntemle gerçekleştirilir:
 - Deep Access (Derin Erişim)
 - Shallow Access (Yüzeysel Erişim)
- Deep/Shallow **access**, Deep/Shallow **binding** ile karıştırılmamalıdır.
 - Binding programın anlamını (semantics) değiştirir.
 - Access ise programda değişkenlere erişim yöntemini (performans) değiştirir.
- Deep access (Derin Erişim)'de Nonlocal değişkenlere erişmek için stack üzerindeki activation record'lar (AR) dinamik zincir boyunca aranır.
- Tüm aktif fonksiyonlar dynamic chain ile bağlıdır.
 - Arama en son çağrılan fonksiyondan başlar.
 - Geri doğru (stack boyunca) ilerler.
 - İlk bulunan değişken kullanılır.

Dinamik Skop

- Dinamik kapsamda aynı anda bir isim için yalnızca bir görünür bağlama vardır.
- Arama, aktivasyon kayıtları içinde zincir boyunca yapılmaz.
- Her isim için “en güncel” değer doğrudan bulunur.

Referans, ilgili ismin stack tepesine gider.



stack tepesi = görünür bağlama

Model 2: Merkezi tablo

Ad	Güncel değer	aktif
A	Sub1.A	1
B	Sub2.B	1
X	Sub2.X	1

- Deep Access: Altprogram bağlantısı hızlı; uzak nonlocal erişimler pahalı.
- Nonlocal erişim hızlı; çağrı/geri dönüş işlemleri daha maliyetli.
- Altprogram çağrıları çok sık ise deep access avantajlı olabilir. Nonlocal referanslar çok sık ve çağrı zincirinde uzak ise shallow access daha uygundur.